JAVA LAB OBSERVATION RECORD

Observation Task 1: Java Programming Fundamentals

Observation Task 2: Object-Oriented Programming in Java

Included: Aim, description, UML diagrams, commented code, terminal-style screenshots, results and detailed viva answers.

Name: _____

Roll Number: _____

TASK 1 — JAVA PROGRAMMING FUNDAMENTALS

1. Aim

To implement Java variables, data types, operators, input/output, conditional statements, loops and parameterized methods through a menu-driven Student Utility Program.

2. Description

The program accepts student name, roll number and three marks (each out of 100), calculates total and percentage, and offers menu choices for even/odd, largest of three, grade, day using switch, multiplication table, numbers from 1 to N, natural-number sum, Fibonacci series, factorial, prime check, maximum of two numbers and circle area.

3. UML / Program Structure

```
StudentUtilityProgram
+ main(String[]): void
+ studentResult(name, roll, marks): void
+ isEven(n): boolean
+ largest(a,b,c): int
+ grade(percentage): String
+ dayName(day): String
+ table(n): void
+ numbers(n): void
+ naturalSum(n): long
+ fibonacci(n): void
+ factorial(n): long
+ prime(n): boolean
+ maximum(a,b): int
+ circleArea(radius): double
Uses Scanner for input; Math for maximum and PI.
```

4. Source Code with Comments

```
import java.util.Scanner;

public class StudentUtilityProgram {
    static void studentResult(String name, int roll, double a, double b, double c) {
        double total = a + b + c;
        System.out.println("Name: " + name + " | Roll No: " + roll);
        System.out.println("Total: " + total + "/300");
        System.out.printf("Percentage: %.2f%%\n", total / 3.0);
    }
    static boolean isEven(int n) { return n % 2 == 0; }
    static int largest(int a, int b, int c) { return Math.max(a, Math.max(b, c)); }
    static String grade(double p) {
        if (p >= 90) return "A";
        else if (p >= 80) return "B";
        else if (p >= 70) return "C";
        else if (p >= 60) return "D";
        else if (p >= 40) return "E";
        else return "F (Fail)";
    }
    static String dayName(int d) {
        switch (d) {
            case 1: return "Monday"; case 2: return "Tuesday";
            case 3: return "Wednesday"; case 4: return "Thursday";
            case 5: return "Friday"; case 6: return "Saturday";
            case 7: return "Sunday";
            default: return "Invalid day";
        }
    }
    static void table(int n) {
        for (int i = 1; i <= 10; i++) System.out.println(n + " x " + i + " = " + n*i);
    }
    static void numbers(int n) {
        int i = 1;
        while (i <= n) { System.out.print(i + " "); i++; }
        System.out.println();
    }
    static long naturalSum(int n) {
        long sum = 0;
        for (int i = 1; i <= n; i++) sum += i;
        return sum;
    }
    static void fibonacci(int n) {
        if (n <= 0) { System.out.println("Enter positive terms."); return; }
        int a = 0, b = 1, count = 0;
        do {
            System.out.print(a + " ");
            int next = a + b; a = b; b = next; count++;
        } while (count < n);
        System.out.println();
    }
    static long factorial(int n) {
        if (n < 0) throw new IllegalArgumentException("Number must be non-negative");
        long result = 1;
        for (int i = 2; i <= n; i++) result *= i;
        return result;
    }
    static boolean prime(int n) {
        if (n < 2) return false;
        for (int i = 2; i <= n / i; i++) if (n % i == 0) return false;
        return true;
    }
    static int maximum(int a, int b) { return a > b ? a : b; }
    static double circleArea(double r) { return Math.PI * r * r; }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;
```

```
        do {
            System.out.println("\n=== STUDENT UTILITY MENU ===");
            System.out.println("1.Student result  2.Even/Odd  3.Largest of 3");
            System.out.println("4.Grade  5.Day  6.Table  7.Numbers 1..N");
            System.out.println("8.Natural sum  9.Fibonacci  10.Factorial");
            System.out.println("11.Prime  12.Maximum of 2  13.Circle area  0.Exit");
            System.out.print("Choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    sc.nextLine();
                    System.out.print("Name: "); String name = sc.nextLine();
                    System.out.print("Roll number: "); int roll = sc.nextInt();
                    System.out.print("Three marks (out of 100): ");
                    studentResult(name, roll, sc.nextDouble(), sc.nextDouble(), sc.nextDouble());
                    break;
                case 2:
                    System.out.print("Integer: "); int n = sc.nextInt();
                    System.out.println(isEven(n) ? "Even" : "Odd"); break;
                case 3:
                    System.out.print("Three integers: ");
                    System.out.println("Largest = " + largest(sc.nextInt(), sc.nextInt(), sc.nextInt())); break;
                case 4:
                    System.out.print("Percentage: "); System.out.println("Grade = " + grade(sc.nextDouble())); break;
                case 5:
                    System.out.print("Day number (1-7): "); System.out.println(dayName(sc.nextInt())); break;
                case 6:
                    System.out.print("Number: "); table(sc.nextInt()); break;
                case 7:
                    System.out.print("N: "); numbers(sc.nextInt()); break;
                case 8:
                    System.out.print("N: "); System.out.println("Sum = " + naturalSum(sc.nextInt())); break;
                case 9:
                    System.out.print("Number of terms: "); fibonacci(sc.nextInt()); break;
                case 10:
                    System.out.print("Non-negative integer: "); System.out.println("Factorial = " + factorial(sc.nextInt())); break;
                case 11:
                    System.out.print("Integer: "); int p = sc.nextInt();
                    System.out.println(prime(p) ? "Prime" : "Not prime"); break;
                case 12:
                    System.out.print("Two integers: "); System.out.println("Maximum = " + maximum(sc.nextInt(), sc.nextInt())); break;
                case 13:
                    System.out.print("Radius: "); System.out.printf("Area = %.2f\n", circleArea(sc.nextDouble())); break;
                case 0: System.out.println("Goodbye!"); break;
                default: System.out.println("Invalid choice.");
            }
        } while (choice != 0);
        sc.close();
    }
}
```

5. Compile and Run

```
javac StudentUtilityProgram.java
java StudentUtilityProgram
```

6. Terminal Output Screenshots

Sample interactive run using representative menu choices:

```
Terminal — Task 1: StudentUtilityProgram

$ java StudentUtilityProgram

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Name: Roll number: Three marks (out of 100): Name: Prasanna | Roll No: 25
Total: 270.0/300
Percentage: 90.00%

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Integer: Even

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Three integers: Largest = 25

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Percentage: Grade = B

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
```

Terminal — Task 1: StudentUtilityProgram

```
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Day number (1-7): Wednesday

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Number: 5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: N: 1 2 3 4 5

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: N: Sum = 15

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
```

Terminal — Task 1: StudentUtilityProgram

```
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Number of terms: 0 1 1 2 3 5 8

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Non-negative integer: Factorial = 120

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Integer: Prime

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Two integers: Maximum = 12

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Radius: Area = 28.27

=== STUDENT UTILITY MENU ===
1.Student result  2.Even/Odd  3.Largest of 3
4.Grade  5.Day  6.Table  7.Numbers 1..N
8.Natural sum  9.Fibonacci  10.Factorial
```

```
Terminal — Task 1: StudentUtilityProgram

11.Prime  12.Maximum of 2  13.Circle area  0.Exit
Choice: Goodbye!
```

7. Observation / Viva Questions and Answers

1. What is the structure of a Java program?

A typical program contains optional package/import statements, a class declaration, fields, constructors and methods. Execution starts from public static void main(String[] args). Example: public class Demo { public static void main(String[] args) { System.out.println("Hello"); } }. The source file name must match the public class name.

2. Primitive vs non-primitive data types.

Primitive types are byte, short, int, long, float, double, char and boolean; they hold simple values. Non-primitive/reference types include String, arrays and classes; variables refer to objects and can be null. Reference types can also provide methods and fields.

3. if-else vs switch.

if-else evaluates boolean expressions and supports ranges or complex conditions, e.g. percentage >= 90. switch compares one expression against case labels and is suitable for fixed choices such as a menu or day number. Traditional switch statements need break/return to avoid fall-through.

4. for vs while vs do-while.

for groups initialization, condition and update and is useful for counted repetition. while checks the condition before its body and can execute zero times. do-while checks after the body, so it executes at least once. This program uses all three styles.

5. What are methods and why are they useful?

A method is a named block that performs a task and can accept parameters and return a value. Methods improve reuse, readability and testing. Example: static int maximum(int a,int b) { return a>b?a:b; }.

6. Formal vs actual parameters.

Formal parameters appear in the method declaration, such as a and b in maximum(int a,int b). Actual parameters/arguments are supplied at the call, such as maximum(4,9). Java passes arguments by value.

8. Result

The Student Utility Program was compiled and executed successfully. It demonstrates Java program structure, decisions, loops and reusable parameterized methods.

TASK 2 — OBJECT-ORIENTED PROGRAMMING IN JAVA

1. Aim

To demonstrate classes, objects, constructors, encapsulation, inheritance, overloading, overriding, polymorphism, abstraction and interfaces in a Library Management System example.

2. Description

Book stores private book details with default and parameterized constructors and getter/setter methods. Student and Faculty inherit Person. AreaCalculator overloads area methods. Car and Bike override Vehicle.display(). Circle and Rectangle implement abstract Shape.draw(), and Report implements Printable. All classes are kept in one file for easy compilation.

3. UML Diagram

```
Book (private fields; constructors; getters/setters; display)
Person <|-- Student
Person <|-- Faculty
AreaCalculator: area(radius), area(length,width), area(base,height) [overloading]
Vehicle <|-- Car [display overridden]
Vehicle <|-- Bike [display overridden]
Shape (abstract draw)
Shape <|-- Circle
Shape <|-- Rectangle
Printable (interface) <|.. Report [implements]
```

4. Source Code with Comments

```
class Book {
    private int bookId;
    private String bookName, author;
    private double price;

    // No-argument constructor delegates to the parameterized constructor.
    public Book() { this(0, "Unknown", "Unknown", 0.0); }

    public Book(int id, String name, String author, double price) {
        this.bookId = id; this.bookName = name; this.author = author;
        setPrice(price);
    }
    public int getBookId() { return bookId; }
    public void setBookId(int id) { bookId = id; }
    public String getBookName() { return bookName; }
    public void setBookName(String name) { bookName = name; }
    public String getAuthor() { return author; }
    public void setAuthor(String value) { author = value; }
    public double getPrice() { return price; }
    public void setPrice(double value) {
        if (value < 0) throw new IllegalArgumentException("Price cannot be negative");
        price = value;
    }
    public void display() {
        System.out.println("Book ID: " + bookId + ", Name: " + bookName
                + ", Author: " + author + ", Price: " + price);
    }
}

class Person {
    protected String name;
    protected int age;
    public Person(String name, int age) { this.name = name; this.age = age; }
    public void displayPerson() { System.out.println("Name: " + name + ", Age: " + age); }
}
class Student extends Person {
    private int rollNo;
    public Student(String name, int age, int rollNo) {
        super(name, age); this.rollNo = rollNo;
    }
    public void displayStudent() { displayPerson(); System.out.println("Roll No: " + rollNo); }
}
class Faculty extends Person {
    private String department;
    public Faculty(String name, int age, String department) {
        super(name, age); this.department = department;
    }
    public void displayFaculty() { displayPerson(); System.out.println("Department: " + department); }
}

// Overloading: same method name, different parameter lists.
class AreaCalculator {
    public double area(double radius) { return Math.PI * radius * radius; }
    public int area(int length, int width) { return length * width; }
    public double area(double base, double height) { return 0.5 * base * height; }
}
class Vehicle {
    public void display() { System.out.println("General vehicle"); }
}
class Car extends Vehicle {
    @Override public void display() { System.out.println("Car: four wheels"); }
}
class Bike extends Vehicle {
    @Override public void display() { System.out.println("Bike: two wheels"); }
}

// Abstraction: subclasses implement the abstract draw() method.
abstract class Shape { public abstract void draw(); }
class Circle extends Shape {
    @Override public void draw() { System.out.println("Drawing a circle"); }
}
class Rectangle extends Shape {
    @Override public void draw() { System.out.println("Drawing a rectangle"); }
```

```
}
interface Printable { void print(); }
class Report implements Printable {
    @Override public void print() { System.out.println("Printing library report"); }
}

public class LibraryManagementSystem {
    public static void main(String[] args) {
        System.out.println("=== BOOK / CONSTRUCTORS / ENCAPSULATION ===");
        Book emptyBook = new Book(); emptyBook.display();
        Book book = new Book(101, "Java Basics", "A. Kumar", 450.0);
        book.display();
        book.setPrice(425.0);
        System.out.println("Updated price: " + book.getPrice());

        System.out.println("\n=== INHERITANCE ===");
        new Student("Ravi", 19, 25).displayStudent();
        new Faculty("Meena", 40, "CSE").displayFaculty();

        System.out.println("\n=== POLYMORPHISM ===");
        AreaCalculator calc = new AreaCalculator();
        System.out.printf("Circle area: %.2f\n", calc.area(3.0));
        System.out.println("Rectangle area: " + calc.area(4, 5));
        System.out.printf("Triangle area: %.2f\n", calc.area(4.0, 6.0));
        Vehicle v1 = new Car(); Vehicle v2 = new Bike();
        v1.display(); v2.display();

        System.out.println("\n=== ABSTRACTION / INTERFACE ===");
        Shape s1 = new Circle(); Shape s2 = new Rectangle();
        s1.draw(); s2.draw();
        Printable report = new Report(); report.print();
        System.out.println("Library OOP demonstration completed.");
    }
}
```

5. Compile and Run

```
javac LibraryManagementSystem.java
java LibraryManagementSystem
```

6. Terminal Output Screenshot

The following screenshots show the output of the OOP demonstration:

Terminal — Task 2: LibraryManagementSystem

```
$ java LibraryManagementSystem
=== BOOK / CONSTRUCTORS / ENCAPSULATION ===
Book ID: 0, Name: Unknown, Author: Unknown, Price: 0.0
Book ID: 101, Name: Java Basics, Author: A. Kumar, Price: 450.0
Updated price: 425.0

=== INHERITANCE ===
Name: Ravi, Age: 19
Roll No: 25
Name: Meena, Age: 40
Department: CSE

=== POLYMORPHISM ===
Circle area: 28.27
Rectangle area: 20
Triangle area: 12.00
Car: four wheels
Bike: two wheels

=== ABSTRACTION / INTERFACE ===
Drawing a circle
Drawing a rectangle
Printing library report
Library OOP demonstration completed.
```

7. Observation / Viva Questions and Answers

1. What is OOP?

Object-Oriented Programming organizes code around objects that combine state (fields) and behavior (methods). Its key ideas are encapsulation, inheritance, polymorphism and abstraction.

2. Class vs object.

A class is a blueprint; an object is an instance created from it. Book is a class, while new Book(101,"Java Basics","A. Kumar",450.0) creates an object.

3. Purpose of constructors.

A constructor initializes an object, has the class name and has no return type. A no-argument constructor takes no arguments; a parameterized constructor accepts values. Constructors can be overloaded.

4. Encapsulation and implementation.

Encapsulation protects object state by hiding fields and providing controlled methods. In Java, private fields plus public getters/setters are common. Book.setPrice() also validates that price is not negative.

5. Overloading vs overriding.

Overloading uses the same method name with different parameter lists and is resolved at compile time; AreaCalculator overloads area(). Overriding replaces an inherited method implementation with the same signature and supports runtime polymorphism; Car and Bike override Vehicle.display().

6. Four pillars of OOP.

Encapsulation: private Book fields and accessors. Inheritance: Student and Faculty extend Person. Polymorphism: overloaded area methods and overridden display methods. Abstraction: abstract Shape defines draw() without specifying the drawing implementation.

7. Abstract class vs interface.

An abstract class can contain instance fields, constructors, concrete and abstract methods; a class can extend only one class. An interface defines a contract and can be implemented alongside other interfaces. Here Shape is abstract, while Printable is implemented by Report.

8. Result

The Library Management System demonstration was compiled and executed successfully. It shows encapsulation, inheritance, constructors, compile-time and runtime polymorphism, abstraction and interfaces.

Viva preparation tip

Understand what each method returns, why fields are private in Book, how super() initializes Person, why overloading is compile-time polymorphism, and how overridden methods are selected at runtime.